DESIGN AND ANALYSIS OF ALGORITHMS – QUICK PRACTICAL NOTES

Practical 1: Fibonacci (Iteration, Recursion) and Tower of Hanoi

Aim: Compute Fibonacci by loop and by recursion; solve Tower of Hanoi recursively.

Algorithm

1. Iterative: a=0,b=1; for i=2..n: c=a+b; a=b; b=c; return b.
2. Recursive: F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2).
3. Hanoi(n,A,B,C): if n==1 move disk; else Hanoi(n-1,A,C,B); move disk n A→C; Hanoi(n-1,B,A,C). Moves = 2^n − 1.

Complexity: Iterative Fibonacci O(n) time, O(1) space. Recursive Fibonacci O(2^n) time, O(n) stack. Hanoi O(2^n) time, O(n) stack.

Code

```java
import java.util.Scanner;

public class Practical1 {

    // 1. ITERATIVE FIBONACCI
    static int fibonacciIterative(int n) {

        int a = 0, b = 1;

        if (n == 0)
            return 0;

        for (int i = 2; i <= n; i++) {

            int c = a + b;
            a = b;
            b = c;
        }

        return b;
    }

    // 2. RECURSIVE FIBONACCI
    static int fibonacciRecursive(int n) {

        if (n == 0)
            return 0;

        if (n == 1)
            return 1;

        return fibonacciRecursive(n - 1)
             + fibonacciRecursive(n - 2);
    }

    // 3. TOWER OF HANOI
    static void towerOfHanoi(int n, char source,
                             char auxiliary, char destination) {

        if (n == 1) {

            System.out.println("Move disk 1 from "
                    + source + " to " + destination);

            return;
        }

        towerOfHanoi(n - 1, source, destination, auxiliary);
```

```
49
50          System.out.println("Move disk " + n + " from "
51                  + source + " to " + destination);
52
53          towerOfHanoi(n - 1, auxiliary, source, destination);
54      }
55
56      // MAIN METHOD
57      public static void main(String[] args) {
58
59          Scanner sc = new Scanner(System.in);
60
61          System.out.println("1. Iterative Fibonacci");
62          System.out.println("2. Recursive Fibonacci");
63          System.out.println("3. Tower of Hanoi");
64
65          System.out.print("Enter choice: ");
66          int choice = sc.nextInt();
67
68          switch (choice) {
69
70              case 1:
71
72                  System.out.print("Enter n: ");
73                  int n1 = sc.nextInt();
74
75                  if (n1 < 0)
76                      System.out.println("Invalid input");
77                  else
78                      System.out.println("Fibonacci = "
79                              + fibonacciIterative(n1));
80
81                  break;
82
83              case 2:
84
85                  System.out.print("Enter n: ");
86                  int n2 = sc.nextInt();
87
88                  if (n2 < 0)
89                      System.out.println("Invalid input");
90                  else
91                      System.out.println("Fibonacci = "
92                              + fibonacciRecursive(n2));
93
94                  break;
95
96              case 3:
97
98                  System.out.print("Enter number of disks: ");
99                  int n3 = sc.nextInt();
100
101                 if (n3 <= 0)
102                     System.out.println("Invalid input");
103                 else
104                     towerOfHanoi(n3, 'A', 'B', 'C');
105
106                 break;
107
108             default:
109                 System.out.println("Invalid choice");
110         }
111
112         sc.close();
113     }
114 }
```

Sample output

```
java Practical1

1. Iterative Fibonacci
2. Recursive Fibonacci
3. Tower of Hanoi
Enter choice: 1
Enter n: 7
Fibonacci = 13
```

```
java Practical1

1. Iterative Fibonacci
2. Recursive Fibonacci
3. Tower of Hanoi
Enter choice: 2
Enter n: 7
Fibonacci = 13
```

```
java Practical1

1. Iterative Fibonacci
2. Recursive Fibonacci
3. Tower of Hanoi
Enter choice: 3
Enter number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Viva

Q1. Base condition?

Ans: Condition that stops recursion (n==0/1; Hanoi n==1).

Q2. Why is recursive Fibonacci slow?

Ans: Same subproblems are recomputed (overlapping subproblems).

Q3. Minimum Hanoi moves for n disks?

Ans: 2^n − 1 (4 disks = 15).

Q4. Missing base condition?

Ans: StackOverflowError.

Q5. Iteration vs recursion?

Ans: Loop vs function calling itself; recursion uses call stack (LIFO).

Practical 2: Fractional Knapsack (Greedy)

Aim: Maximise value in a knapsack where items can be taken in fractions.

Algorithm

1. ratio = value/weight for each item.
2. Sort items by ratio, descending.
3. For each item: if weight ≤ remaining, take all; else take fraction = remaining/weight and stop.

Complexity: Time O(n log n) (sorting dominates), space O(n). Greedy is optimal here.

Code

```java
import java.util.*;

public class FractionalKnapsack {

    static class Item {
        int value, weight, index;
        double ratio;

        Item(int value, int weight, int index) {
            this.value = value;
            this.weight = weight;
            this.index = index;
            this.ratio = (double) value / weight;
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of items: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Invalid number of items");
            sc.close();
            return;
        }

        Item[] items = new Item[n];

        for (int i = 0; i < n; i++) {

            System.out.print("Enter value of item " + (i + 1) + ": ");
            int value = sc.nextInt();

            System.out.print("Enter weight of item " + (i + 1) + ": ");
            int weight = sc.nextInt();

            if (value < 0 || weight <= 0) {
                System.out.println("Invalid value or weight");
                sc.close();
                return;
            }

            items[i] = new Item(value, weight, i + 1);
        }

```

```
        System.out.print("Enter knapsack capacity: ");
        int capacity = sc.nextInt();

        if (capacity < 0) {
            System.out.println("Invalid capacity");
            sc.close();
            return;
        }

        // Sort by value/weight ratio in descending order
        Arrays.sort(items, (a, b) ->
            Double.compare(b.ratio, a.ratio)
        );

        double totalValue = 0;
        double remainingCapacity = capacity;

        System.out.println("\nSelected Items:");

        for (int i = 0; i < n; i++) {

            if (remainingCapacity == 0)
                break;

            if (items[i].weight <= remainingCapacity) {

                remainingCapacity -= items[i].weight;
                totalValue += items[i].value;

                System.out.println("Item " + items[i].index
                        + " -> 100% selected");

            } else {

                double fraction =
                    remainingCapacity / items[i].weight;

                totalValue += items[i].value * fraction;

                System.out.printf("Item %d -> %.2f%% selected%n",
                        items[i].index, fraction * 100);

                remainingCapacity = 0;
            }
        }

        System.out.printf("\nMaximum value = %.2f%n", totalValue);

        sc.close();
    }
}
```

Sample output

```
java FractionalKnapsack
```

```
Enter number of items: 3
Enter value of item 1: 60
Enter weight of item 1: 10
Enter value of item 2: 100
Enter weight of item 2: 20
Enter value of item 3: 120
Enter weight of item 3: 30
Enter knapsack capacity: 50

Selected Items:
Item 1 -> 100% selected
Item 2 -> 100% selected
Item 3 -> 66.67% selected

Maximum value = 240.00
```

Viva

Q1. Why sort by ratio?

Ans: Highest value per unit weight first gives optimum.

Q2. Fractional vs 0/1?

Ans: Fractional allows fractions (greedy); 0/1 does not (DP).

Q3. Why store index?

Ans: Sorting changes positions; index keeps original item number.

Q4. Why (double) cast?

Ans: Avoid integer division in value/weight.

Practical 3: Huffman Encoding (Greedy)

Aim: Build Huffman tree, generate codes, encode and decode text.

Algorithm

1. Count frequency of each character.
2. Put leaf nodes in a min-priority queue (by frequency).
3. Repeat: remove two smallest, make parent (sum), insert back, until one node (root) remains.
4. Left edge = 0, right edge = 1; root-to-leaf path = code. Decode by walking the tree bit by bit.

Complexity: Build O(k log k) (k distinct chars); encode/decode O(m) (m = encoded bits); space O(k).

Code

```java
import java.util.*;

public class HuffmanEncoding {

    static class Node {

        char ch;
        int freq;
        int order;
        Node left, right;

        Node(char ch, int freq, int order) {
            this.ch = ch;
            this.freq = freq;
            this.order = order;
        }

        Node(int freq, Node left, Node right, int order) {
            this.freq = freq;
            this.left = left;
            this.right = right;
            this.order = order;
        }
    }

    static Node root;
    static int count = 0;

    static Map<Character, String> codes = new TreeMap<>();

    // 1. BUILD HUFFMAN TREE
    static void buildTree(String text) {

        Map<Character, Integer> freq = new TreeMap<>();

        for (char c : text.toCharArray()) {
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        }

        PriorityQueue<Node> pq = new PriorityQueue<>(
            Comparator.comparingInt((Node n) -> n.freq)
                      .thenComparingInt(n -> n.order)
        );

        count = 0;

        for (Map.Entry<Character, Integer> e : freq.entrySet()) {
            pq.add(new Node(e.getKey(), e.getValue(), count++));
```

```
        }

        if (pq.size() == 1) {

            Node only = pq.poll();
            root = new Node(only.freq, only, null, count++);

        } else {

            while (pq.size() > 1) {

                Node left = pq.poll();
                Node right = pq.poll();

                Node parent = new Node(
                    left.freq + right.freq,
                    left, right, count++
                );

                pq.add(parent);
            }

            root = pq.poll();
        }

        codes.clear();

        generateCodes(root, "");
    }

    // 2. GENERATE HUFFMAN CODES
    static void generateCodes(Node node, String code) {

        if (node == null)
            return;

        if (node.left == null && node.right == null) {

            codes.put(node.ch, code.isEmpty() ? "0" : code);
            return;
        }

        generateCodes(node.left, code + "0");

        generateCodes(node.right, code + "1");
    }

    // 3. ENCODE TEXT
    static String encode(String text) {

        StringBuilder result = new StringBuilder();

        for (char c : text.toCharArray()) {
            result.append(codes.get(c));
        }

        return result.toString();
    }

    // 4. DECODE TEXT
    static String decode(String encoded) {

        StringBuilder result = new StringBuilder();

```

```
        Node current = root;

        for (char bit : encoded.toCharArray()) {

            if (bit == '0')
                current = current.left;

            else if (bit == '1')
                current = current.right;

            else
                return null;

            if (current == null)
                return null;

            if (current.left == null && current.right == null) {

                result.append(current.ch);

                current = root;
            }
        }

        if (current != root)
            return null;

        return result.toString();
    }

    // 5. DISPLAY HUFFMAN CODES
    static void displayCodes() {

        System.out.println("\nHuffman Codes:");

        for (Map.Entry<Character, String> e : codes.entrySet()) {

            if (e.getKey() == ' ')
                System.out.println("[space] : " + e.getValue());

            else
                System.out.println(e.getKey() + " : " + e.getValue());
        }
    }

    // MAIN METHOD
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter text: ");
        String text = sc.nextLine();

        if (text.isEmpty()) {

            System.out.println("Text cannot be empty.");
            sc.close();
            return;
        }

        buildTree(text);

        int choice;

```

```
        do {

            System.out.println("\n--- HUFFMAN ENCODING ---");
            System.out.println("1. Build Huffman Tree");
            System.out.println("2. Encode Text");
            System.out.println("3. Decode Text");
            System.out.println("4. Exit");

            System.out.print("Enter choice: ");
            choice = sc.nextInt();

            switch (choice) {

                case 1:

                    buildTree(text);

                    System.out.println("Huffman Tree built successfully.");

                    displayCodes();

                    break;

                case 2:

                    String encoded = encode(text);

                    System.out.println("Original Text: " + text);
                    System.out.println("Encoded Text: " + encoded);

                    displayCodes();

                    break;

                case 3:

                    String encodedText = encode(text);

                    String decoded = decode(encodedText);

                    System.out.println("Encoded Text: " + encodedText);
                    System.out.println("Decoded Text: " + decoded);

                    break;

                case 4:

                    System.out.println("Program terminated.");

                    break;

                default:

                    System.out.println("Invalid choice.");
            }

        } while (choice != 4);

        sc.close();
    }
}
```

Sample output

java HuffmanEncoding

```
Enter text: AAABBCCCC

--- HUFFMAN ENCODING ---
1. Build Huffman Tree
2. Encode Text
3. Decode Text
4. Exit
Enter choice: 1
Huffman Tree built successfully.

Huffman Codes:
A : 11
B : 10
C : 0

--- HUFFMAN ENCODING ---
1. Build Huffman Tree
2. Encode Text
3. Decode Text
4. Exit
Enter choice: 2
Original Text: AAABBCCCC
Encoded Text: 11111110100000

Huffman Codes:
A : 11
B : 10
C : 0

--- HUFFMAN ENCODING ---
1. Build Huffman Tree
2. Encode Text
3. Decode Text
4. Exit
Enter choice: 3
Encoded Text: 11111110100000
Decoded Text: AAABBCCCC

--- HUFFMAN ENCODING ---
1. Build Huffman Tree
2. Encode Text
3. Decode Text
4. Exit
Enter choice: 4
Program terminated.
```

Viva

Q1. Prefix-free code?

Ans: No code is a prefix of another, so decoding is unambiguous.

Q2. Why greedy?

Ans: Always merges the two least frequent nodes.

Q3. Is it lossless?

Ans: Yes, original text is recovered exactly.

Q4. Why PriorityQueue?

Ans: Gets minimum-frequency nodes in O(log k).

Q5. Single distinct character?

Ans: Wrapped in a root; code is 0.

Practical 4: 0/1 Knapsack (Dynamic Programming)

Aim: Maximise value when each item is taken fully or not at all.

Algorithm

1. dp[i][w] = best value using first i items, capacity w; dp[0][w]=dp[i][0]=0.
2. If weight[i-1] ≤ w: dp[i][w] = max(dp[i-1][w], value[i-1] + dp[i-1][w-weight[i-1]]); else dp[i][w] = dp[i-1][w].
3. Answer dp[n][W]. Backtrack from (n,W): if dp[i][w] ≠ dp[i-1][w] item i is chosen, w −= weight[i-1].

Complexity: Time O(n·W), space O(n·W) (pseudo-polynomial). Brute force is O(2^n).

Code

```java
import java.util.Scanner;

public class KnapsackDP {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of items: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Number of items must be positive.");
            sc.close();
            return;
        }

        int[] value = new int[n];
        int[] weight = new int[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter value of item " + (i + 1) + ": ");
            value[i] = sc.nextInt();

            System.out.print("Enter weight of item " + (i + 1) + ": ");
            weight[i] = sc.nextInt();

            if (value[i] < 0 || weight[i] <= 0) {
                System.out.println("Invalid value or weight.");
                sc.close();
                return;
            }
        }

        System.out.print("Enter knapsack capacity: ");
        int W = sc.nextInt();

        if (W < 0) {
            System.out.println("Capacity cannot be negative.");
            sc.close();
            return;
        }

        int[][] dp = new int[n + 1][W + 1];

        for (int i = 1; i <= n; i++) {
            for (int w = 0; w <= W; w++) {
                if (weight[i - 1] <= w) {
                    dp[i][w] = Math.max(
                        dp[i - 1][w],
```

```
                        value[i - 1] + dp[i - 1][w - weight[i - 1]]
                    );
                } else {
                    dp[i][w] = dp[i - 1][w];
                }
            }
        }

        System.out.println("\nMaximum value = " + dp[n][W]);

        System.out.println("Selected items:");

        int w = W;

        for (int i = n; i > 0; i--) {
            if (dp[i][w] != dp[i - 1][w]) {
                System.out.println("Item " + i);
                w = w - weight[i - 1];
            }
        }

        sc.close();
    }
}
```

Sample output

```
java KnapsackDP
Enter number of items: 3
Enter value of item 1: 40
Enter weight of item 1: 20
Enter value of item 2: 60
Enter weight of item 2: 20
Enter value of item 3: 80
Enter weight of item 3: 40
Enter knapsack capacity: 50

Maximum value = 100
Selected items:
Item 2
Item 1
```

Viva

Q1. What does dp[i][w] mean?

Ans: Max value with first i items and capacity w.

Q2. Why n+1 rows, W+1 columns?

Ans: To include zero items and zero capacity.

Q3. Why greedy fails?

Ans: e.g. (60,10),(100,20),(120,30), W=50: greedy 160, DP 220.

Q4. How are items found?

Ans: Backtracking: dp[i][w] ≠ dp[i-1][w] means item i selected.

Practical 5: N-Queens (Backtracking)

Aim: Place N queens on an N×N board so none attack each other.

Algorithm

1. solve(row): if row==N return true.
2. For each col: if isSafe(row,col) place queen; if solve(row+1) return true; else remove queen (backtrack).
3. isSafe checks column and the upper-left and upper-right diagonals (rows above only).
4. Return false if no column works.

Complexity: Time about O(N!) (O(N·N!) with safety checks), space O(N^2).

Code

```java
import java.util.Scanner;

public class Nqueens {

    static int[][] board;
    static int n;

    static boolean isSafe(int row, int col) {

        for (int i = 0; i < row; i++) {
            if (board[i][col] == 1)
                return false;
        }

        for (int i = row - 1, j = col - 1;
             i >= 0 && j >= 0; i--, j--) {

            if (board[i][j] == 1)
                return false;
        }

        for (int i = row - 1, j = col + 1;
             i >= 0 && j < n; i--, j++) {

            if (board[i][j] == 1)
                return false;
        }

        return true;
    }

    static boolean solveNQueens(int row) {

        if (row == n)
            return true;

        for (int col = 0; col < n; col++) {

            if (isSafe(row, col)) {

                board[row][col] = 1;

                if (solveNQueens(row + 1))
                    return true;

                board[row][col] = 0;
            }
        }
```

```
49
50          return false;
51      }
52
53      public static void main(String[] args) {
54
55          Scanner sc = new Scanner(System.in);
56
57          System.out.print("Enter the value of N: ");
58          n = sc.nextInt();
59
60          if (n < 1) {
61              System.out.println("Invalid N");
62              sc.close();
63              return;
64          }
65
66          board = new int[n][n];
67
68          if (solveNQueens(0)) {
69
70              System.out.println("Solution Matrix:");
71
72              for (int i = 0; i < n; i++) {
73
74                  for (int j = 0; j < n; j++) {
75                      System.out.print(board[i][j] + " ");
76                  }
77
78                  System.out.println();
79              }
80
81          } else {
82              System.out.println("No solution exists.");
83          }
84
85          sc.close();
86      }
87  }
```

Sample output

java Nqueens

```
Enter the value of N: 4
Solution Matrix:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
```

java Nqueens

```
Enter the value of N: 3
No solution exists.
```

Viva

Q1. What is backtracking?

Ans: Choose, explore, undo if it fails.

Q2. Why only upper rows checked?

Ans: Queens are placed top to bottom; lower rows are empty.

Q3. Solutions for N=2,3?

Ans: None.

Q4. Base condition?

Ans: row == n: all queens placed.